# Chloy_2447116_P5

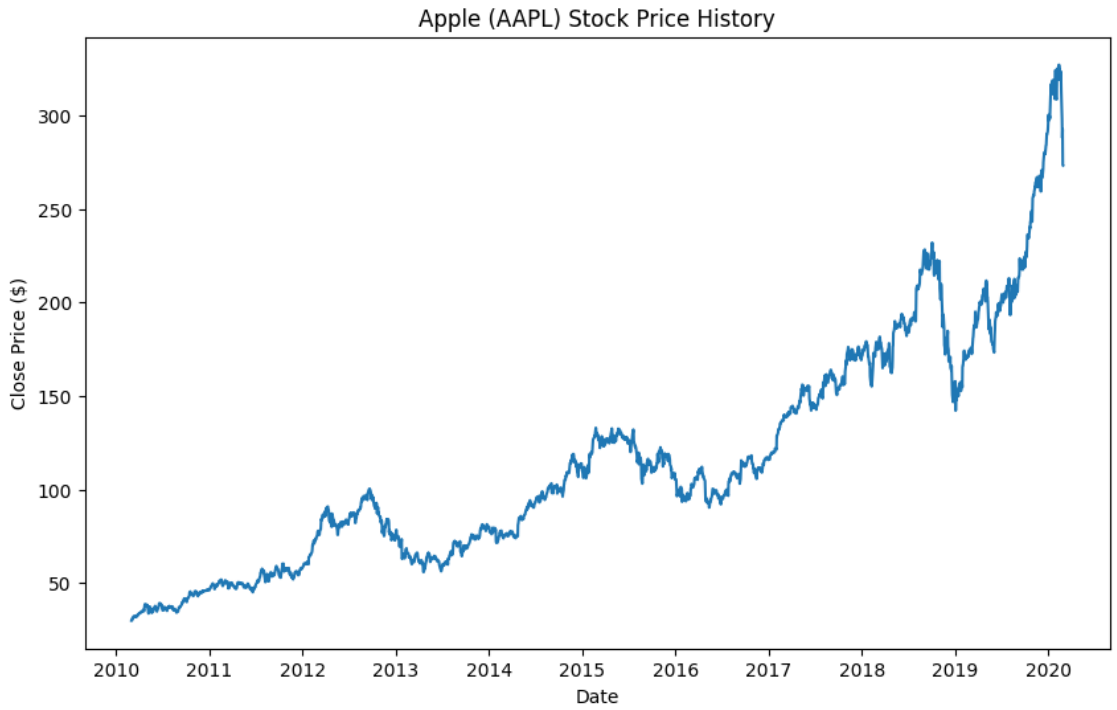
November 20, 2025

[1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_absolute_error, mean_squared_error
import math

# Set random seed for reproducibility
tf.random.set_seed(42)
np.random.seed(42)
```

```
2025-11-20 11:44:22.788583: I external/local_xla/xla/tsl/cuda/cudart_stub.cc:31]
Could not find cuda drivers on your machine, GPU will not be used.
2025-11-20 11:44:24.046106: I tensorflow/core/platform/cpu_feature_guard.cc:210]
This TensorFlow binary is optimized to use available CPU instructions in
performance-critical operations.
To enable the following instructions: AVX2 FMA, in other operations, rebuild
TensorFlow with the appropriate compiler flags.
2025-11-20 11:44:27.577743: I external/local_xla/xla/tsl/cuda/cudart_stub.cc:31]
Could not find cuda drivers on your machine, GPU will not be used.
```

[2]:
```
# Load the dataset
df = pd.read_csv('HistoricalQuotes.csv')

# 1. Clean Column Names: Remove leading/trailing spaces
df.columns = df.columns.str.strip()

# 2. Clean Data: Remove '$' from the 'Close/Last' column and convert to float
# The dataset column is named 'Close/Last' based on the file provided
df['Close/Last'] = df['Close/Last'].astype(str).str.replace('$', '').
 ↪astype(float)

# 3. Sort by Date: The CSV is usually in reverse chronological order, we need␣
 ↪it oldest to newest
df['Date'] = pd.to_datetime(df['Date'])
df = df.sort_values('Date')
```

```
# Focus on the target variable
data = df[['Close/Last']].values

# Visualize the raw data
plt.figure(figsize=(10, 6))
plt.plot(df['Date'], data)
plt.title('Apple (AAPL) Stock Price History')
plt.xlabel('Date')
plt.ylabel('Close Price ($)')
plt.show()
```




[3]:
```
# Normalize the data to 0-1 range
scaler = MinMaxScaler(feature_range=(0, 1))
scaled_data = scaler.fit_transform(data)

# Split into Training (80%) and Testing (20%)
# Note: For time series, we split sequentially, not randomly
train_size = int(len(scaled_data) * 0.8)
train_data = scaled_data[:train_size]
test_data = scaled_data[train_size:]

print(f"Total samples: {len(scaled_data)}")
print(f"Training samples: {len(train_data)}")
print(f"Testing samples: {len(test_data)}")
```

```
Total samples: 2518
Training samples: 2014
Testing samples: 504
```

[4]:
```python
def create_sequences(dataset, seq_length=60):
    X, y = [], []
    for i in range(len(dataset) - seq_length):
        # Create a sequence of 60 days
        X.append(dataset[i:i+seq_length, 0])
        # The target is the price of the next day (61st day)
        y.append(dataset[i+seq_length, 0])
    return np.array(X), np.array(y)

# Define sequence length
SEQ_LENGTH = 60

# Create sequences for training
X_train, y_train = create_sequences(train_data, SEQ_LENGTH)

# Create sequences for testing
# We need the last 60 days of training data to predict the first day of test␣
 ↪data
# So we concatenate part of train data to test data before sequencing
inputs = scaled_data[len(scaled_data) - len(test_data) - SEQ_LENGTH:]
X_test, y_test = create_sequences(inputs, SEQ_LENGTH)

# Reshape input to be [samples, time steps, features] which is required for RNN
X_train = np.reshape(X_train, (X_train.shape[0], X_train.shape[1], 1))
X_test = np.reshape(X_test, (X_test.shape[0], X_test.shape[1], 1))

print(f"X_train shape: {X_train.shape}")
print(f"X_test shape: {X_test.shape}")
```

```
X_train shape: (1954, 60, 1)
X_test shape: (504, 60, 1)
```

[5]:
```python
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import SimpleRNN, Dense

model = Sequential()

# RNN Layer with 50 units
# input_shape = (time_steps, features) -> (60, 1)
model.add(SimpleRNN(50, return_sequences=False, input_shape=(X_train.shape[1],␣
 ↪1)))

# Dense Layer with 1 unit for regression output
model.add(Dense(1))
```

```
# Compile the model
model.compile(optimizer='adam', loss='mean_squared_error')

model.summary()
```

```
2025-11-20 11:44:30.341967: E
external/local_xla/xla/stream_executor/cuda/cuda_platform.cc:51] failed call to
cuInit: INTERNAL: CUDA error: Failed call to cuInit: UNKNOWN ERROR (303)
/home/chloycosta/Documents/College_code/Sem_5/NNDL/.venv/lib64/python3.11/site-
packages/keras/src/layers/rnn/rnn.py:199: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential models,
prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(**kwargs)
```

**Model: "sequential"**

| Layer (type) | Output Shape | Param # |
|---|---|---|
| simple_rnn (SimpleRNN) | (None, 50) | 2,600 |
| dense (Dense) | (None, 1) | 51 |

**Total params:** 2,651 (10.36 KB)

**Trainable params:** 2,651 (10.36 KB)

**Non-trainable params:** 0 (0.00 B)

[6]:
```
# Train for 50 epochs with batch size 32
history = model.fit(
    X_train, y_train,
    batch_size=32,
    epochs=50,
    validation_data=(X_test, y_test),
    verbose=1
)
```

```
Epoch 1/50
62/62             3s 16ms/step -
loss: 0.0022 - val_loss: 0.0065
Epoch 2/50
62/62             1s 11ms/step -
loss: 1.3146e-04 - val_loss: 0.0069
```

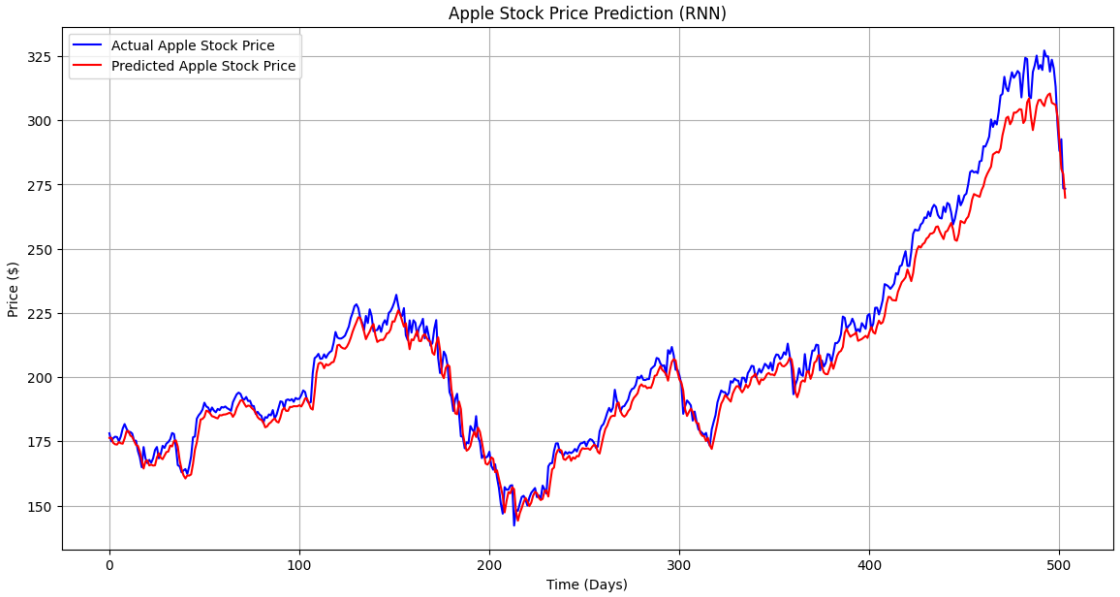
```
Epoch 3/50
62/62             1s 11ms/step -
loss: 1.2501e-04 - val_loss: 0.0058
Epoch 4/50
62/62             1s 11ms/step -
loss: 1.0903e-04 - val_loss: 0.0046
Epoch 5/50
62/62             1s 12ms/step -
loss: 9.5532e-05 - val_loss: 0.0040
Epoch 6/50
62/62             1s 11ms/step -
loss: 8.7595e-05 - val_loss: 0.0034
Epoch 7/50
62/62             1s 11ms/step -
loss: 7.8668e-05 - val_loss: 0.0028
Epoch 8/50
62/62             1s 12ms/step -
loss: 7.3828e-05 - val_loss: 0.0025
Epoch 9/50
62/62             1s 11ms/step -
loss: 7.0800e-05 - val_loss: 0.0022
Epoch 10/50
62/62             1s 11ms/step -
loss: 6.8531e-05 - val_loss: 0.0020
Epoch 11/50
62/62             1s 11ms/step -
loss: 6.6260e-05 - val_loss: 0.0019
Epoch 12/50
62/62             1s 11ms/step -
loss: 6.3867e-05 - val_loss: 0.0017
Epoch 13/50
62/62             1s 10ms/step -
loss: 6.1566e-05 - val_loss: 0.0016
Epoch 14/50
62/62             1s 11ms/step -
loss: 5.9504e-05 - val_loss: 0.0015
Epoch 15/50
62/62             1s 11ms/step -
loss: 5.7716e-05 - val_loss: 0.0014
Epoch 16/50
62/62             1s 11ms/step -
loss: 5.6182e-05 - val_loss: 0.0013
Epoch 17/50
62/62             1s 11ms/step -
loss: 5.4859e-05 - val_loss: 0.0012
Epoch 18/50
62/62             1s 11ms/step -
loss: 5.3708e-05 - val_loss: 0.0012
```

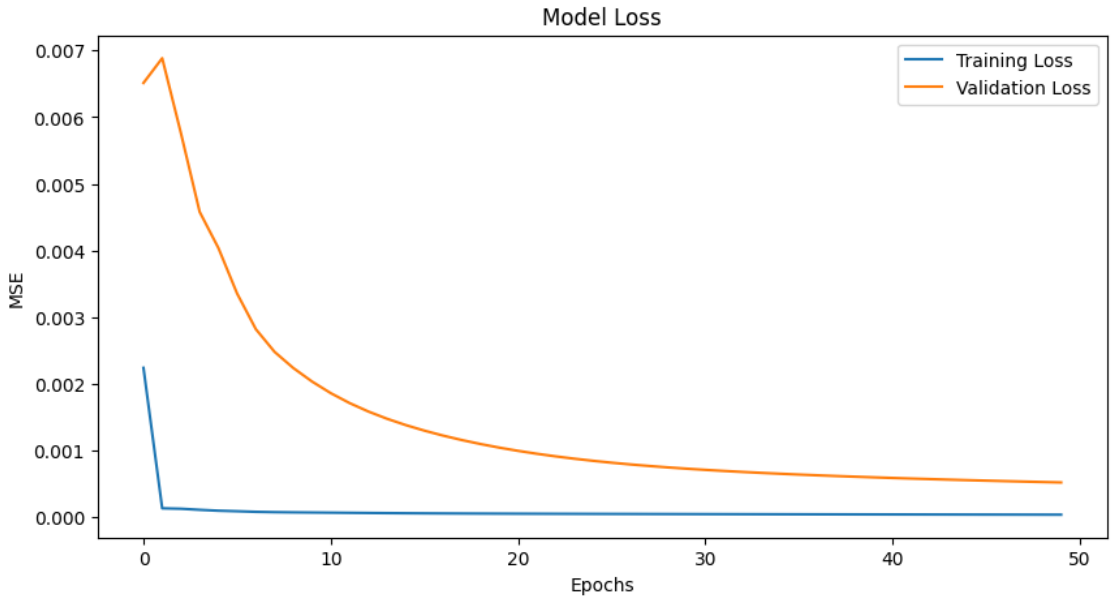
```
Epoch 19/50
62/62             1s 11ms/step -
loss: 5.2689e-05 - val_loss: 0.0011
Epoch 20/50
62/62             1s 11ms/step -
loss: 5.1774e-05 - val_loss: 0.0010
Epoch 21/50
62/62             1s 12ms/step -
loss: 5.0937e-05 - val_loss: 9.9372e-04
Epoch 22/50
62/62             1s 11ms/step -
loss: 5.0160e-05 - val_loss: 9.5023e-04
Epoch 23/50
62/62             1s 11ms/step -
loss: 4.9429e-05 - val_loss: 9.1112e-04
Epoch 24/50
62/62             1s 11ms/step -
loss: 4.8733e-05 - val_loss: 8.7595e-04
Epoch 25/50
62/62             1s 11ms/step -
loss: 4.8064e-05 - val_loss: 8.4427e-04
Epoch 26/50
62/62             1s 11ms/step -
loss: 4.7416e-05 - val_loss: 8.1574e-04
Epoch 27/50
62/62             1s 11ms/step -
loss: 4.6786e-05 - val_loss: 7.8996e-04
Epoch 28/50
62/62             1s 11ms/step -
loss: 4.6171e-05 - val_loss: 7.6660e-04
Epoch 29/50
62/62             1s 11ms/step -
loss: 4.5570e-05 - val_loss: 7.4538e-04
Epoch 30/50
62/62             1s 11ms/step -
loss: 4.4980e-05 - val_loss: 7.2602e-04
Epoch 31/50
62/62             1s 11ms/step -
loss: 4.4403e-05 - val_loss: 7.0829e-04
Epoch 32/50
62/62             1s 11ms/step -
loss: 4.3837e-05 - val_loss: 6.9200e-04
Epoch 33/50
62/62             1s 11ms/step -
loss: 4.3283e-05 - val_loss: 6.7696e-04
Epoch 34/50
62/62             1s 11ms/step -
loss: 4.2741e-05 - val_loss: 6.6300e-04
```

```
Epoch 35/50
62/62             1s 11ms/step -
loss: 4.2211e-05 - val_loss: 6.5002e-04
Epoch 36/50
62/62             1s 11ms/step -
loss: 4.1694e-05 - val_loss: 6.3788e-04
Epoch 37/50
62/62             1s 11ms/step -
loss: 4.1189e-05 - val_loss: 6.2648e-04
Epoch 38/50
62/62             1s 11ms/step -
loss: 4.0697e-05 - val_loss: 6.1574e-04
Epoch 39/50
62/62             1s 11ms/step -
loss: 4.0219e-05 - val_loss: 6.0559e-04
Epoch 40/50
62/62             1s 11ms/step -
loss: 3.9754e-05 - val_loss: 5.9596e-04
Epoch 41/50
62/62             1s 12ms/step -
loss: 3.9302e-05 - val_loss: 5.8679e-04
Epoch 42/50
62/62             1s 11ms/step -
loss: 3.8863e-05 - val_loss: 5.7803e-04
Epoch 43/50
62/62             1s 11ms/step -
loss: 3.8437e-05 - val_loss: 5.6965e-04
Epoch 44/50
62/62             1s 11ms/step -
loss: 3.8024e-05 - val_loss: 5.6163e-04
Epoch 45/50
62/62             1s 11ms/step -
loss: 3.7624e-05 - val_loss: 5.5392e-04
Epoch 46/50
62/62             1s 11ms/step -
loss: 3.7236e-05 - val_loss: 5.4650e-04
Epoch 47/50
62/62             1s 12ms/step -
loss: 3.6859e-05 - val_loss: 5.3937e-04
Epoch 48/50
62/62             1s 12ms/step -
loss: 3.6494e-05 - val_loss: 5.3249e-04
Epoch 49/50
62/62             1s 13ms/step -
loss: 3.6141e-05 - val_loss: 5.2588e-04
Epoch 50/50
62/62             1s 12ms/step -
loss: 3.5798e-05 - val_loss: 5.1950e-04
```

[7]:
```
# Get predictions
predictions = model.predict(X_test)

# Inverse transform to get back original price scale
predictions = scaler.inverse_transform(predictions)
y_test_actual = scaler.inverse_transform(y_test.reshape(-1, 1))

# Plotting the results
plt.figure(figsize=(14, 7))
plt.plot(y_test_actual, color='blue', label='Actual Apple Stock Price')
plt.plot(predictions, color='red', label='Predicted Apple Stock Price')
plt.title('Apple Stock Price Prediction (RNN)')
plt.xlabel('Time (Days)')
plt.ylabel('Price ($)')
plt.legend()
plt.grid(True)
plt.show()

# Plot Training vs Validation Loss
plt.figure(figsize=(10, 5))
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Model Loss')
plt.xlabel('Epochs')
plt.ylabel('MSE')
plt.legend()
plt.show()
```

```
16/16                0s 18ms/step
```






[8]:
```
# Calculate Metrics
mae = mean_absolute_error(y_test_actual, predictions)
rmse = math.sqrt(mean_squared_error(y_test_actual, predictions))

print("--- Model Evaluation ---")
print(f"Mean Absolute Error (MAE): ${mae:.4f}")
print(f"Root Mean Squared Error (RMSE): ${rmse:.4f}")
```

```
--- Model Evaluation ---
Mean Absolute Error (MAE): $5.2617
Root Mean Squared Error (RMSE): $6.7777
```